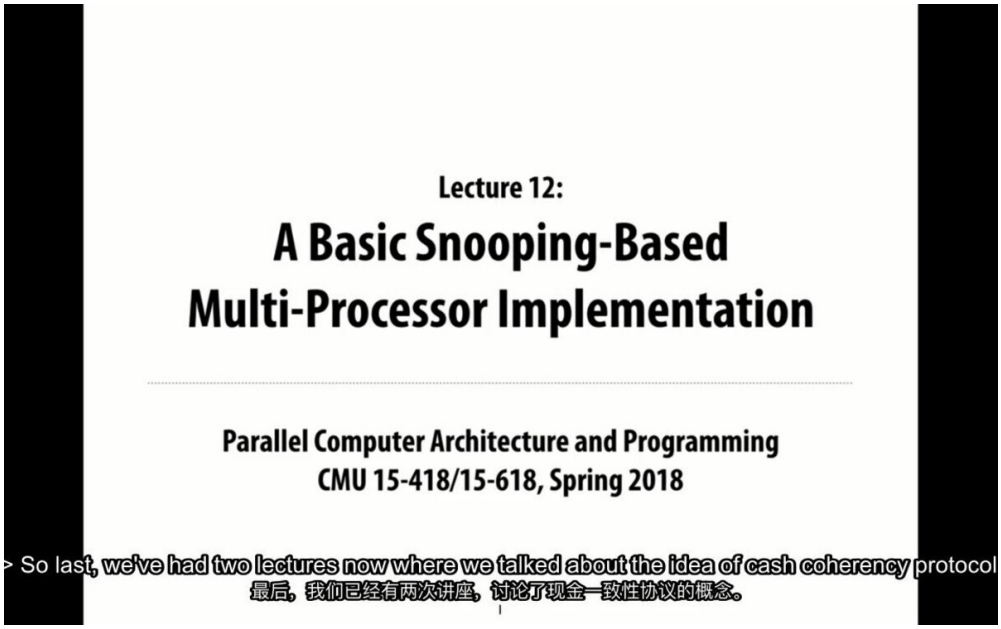
课程笔记

监听式一致性的实现：包含属性、活锁与总线事务

CMU 15-418/618 并行计算机架构与编程 · Lecture 14（Spring 2018）

lecture-to-notes

2026 年 10 月 29 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 17 分 8 秒
视频链接：`https://www.bilibili.com/video/BV1Wp4y1J78s/`

目录

1 从状态图到硬件 **2**

1.1 为什么值得单独讲 2

1.2 这是一类普遍问题 3

1.3 本章小结 3

2 回顾：MESI 与多级缓存的现实 **4**

2.1 状态的含义 4

2.2 多级缓存的现实 5

2.3 本章小结 5

3 包含属性：L2 如何知道 L1 有什么 **5**

3.1 包含属性的定义 5

3.2 有些实现放弃了包含性 6

3.3 LRU 不保证包含性 6

3.4 L1 写命中与额外状态位 7

3.5 本章小结 8

4 三个目标与三种失败 **8**

4.1 目标 8

4.2 死锁 9

4.3 活锁 10

4.4 指数退避 10

4.5 饥饿 11

4.6 本章小结 11

5 简化假设与总线事务 **12**

5.1 假设 12

5.2 总线事务的步骤 13

5.3 本章小结 13

6 多处理器控制器与 Snoop Pending **14**

6.1 单处理器逻辑与多处理的差别 14

6.2 核心挑战 14

6.3 Snoop Pending 15

6.4 本章小结 15

7 数据从哪来：事务分裂 **16**

7.1 两种数据来源 16

7.2 这是一个设计决策 16

7.3 本章小结 16

8 总结与延伸 **16**

1　从状态图到硬件

前两讲讲了一致性协议：先是基于总线（集中式）的，然后是目录式的。这一讲深入总线式的实现细节，目的是让大家体会真正把它做出来时的复杂性。

1.1　为什么值得单独讲

讲者的开场很坦诚：他认识工业界以此为生的人，这些协议确实极其复杂、极难做对。

而”如何验证这类协议”本身就是一个有趣的研究方向，而不是只靠跑一堆模拟。他提到这是形式验证工具取得相当成功的一个领域，部分原因在于用别的方法根本做不到。

协议验证为何必须形式化

一致性协议的出错场景往往需要特定的消息交错才触发，随机模拟很难覆盖。而形式验证把「所有可达状态」作为检查对象，因此能证明不存在违反不变量的交错。这与测试的关系类似「证明」与「抽样」的区别。

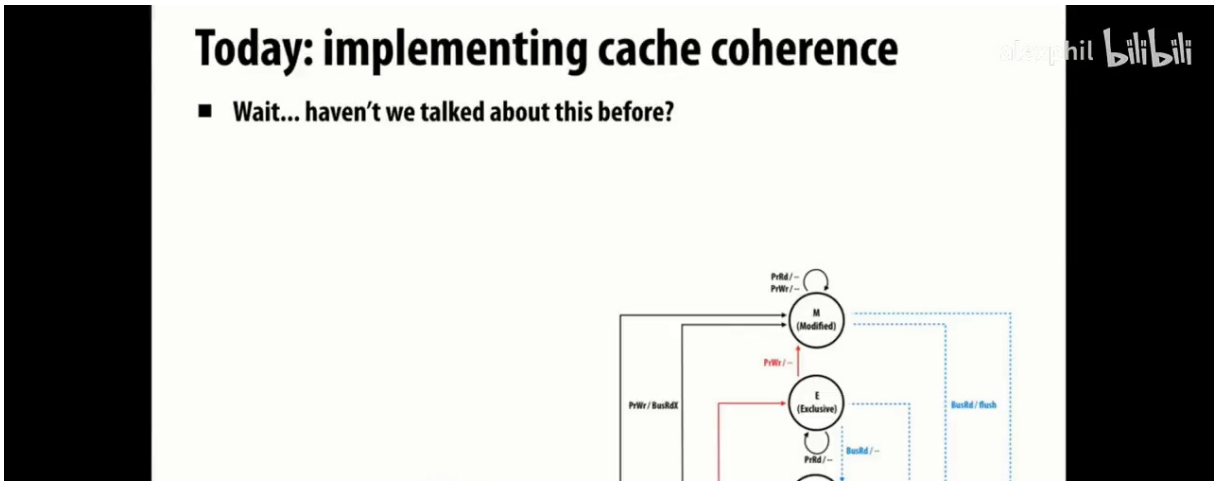


图 1: 今日主题：实现缓存一致性，状态图之外的细节[1]

[1] 视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。



图 2: 今日概念的范围远超硬件实现本身[2]

1.2 这是一类普遍问题

讲者强调这些思想不只属于缓存协议。这是你走向社会、编写系统时都会遇到的一般问题：只要系统涉及并发、有大量事情通过不同主体的独立行动同时发生，就有很多地方会出错。

因此需要跟踪它们，并且有一套系统化的方法。

状态图描述的是规则，不是实现

状态图能告诉你" 从 M 到 I 需要发什么消息"，但不能告诉你" 这条消息发出去时另一个缓存正在做什么"。实现层面的困难来自并发交错、多级层次、仲裁与超时，这些都不在状态图上。因此协议正确性论证必须针对实现，而不能只针对状态机。

1.3 本章小结

这一讲深入总线式一致性的实现细节。工业界认为这类协议极其复杂、极难做对，形式验证在此领域相当成功。这些思想不限于缓存协议：任何有并发、多个独立主体的系统都有大量出错方式，需要系统化方法。

下一章先回顾协议状态，再看真实机器的多级缓存。

[2]视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

2 回顾：MESI 与多级缓存的现实

2.1 状态的含义

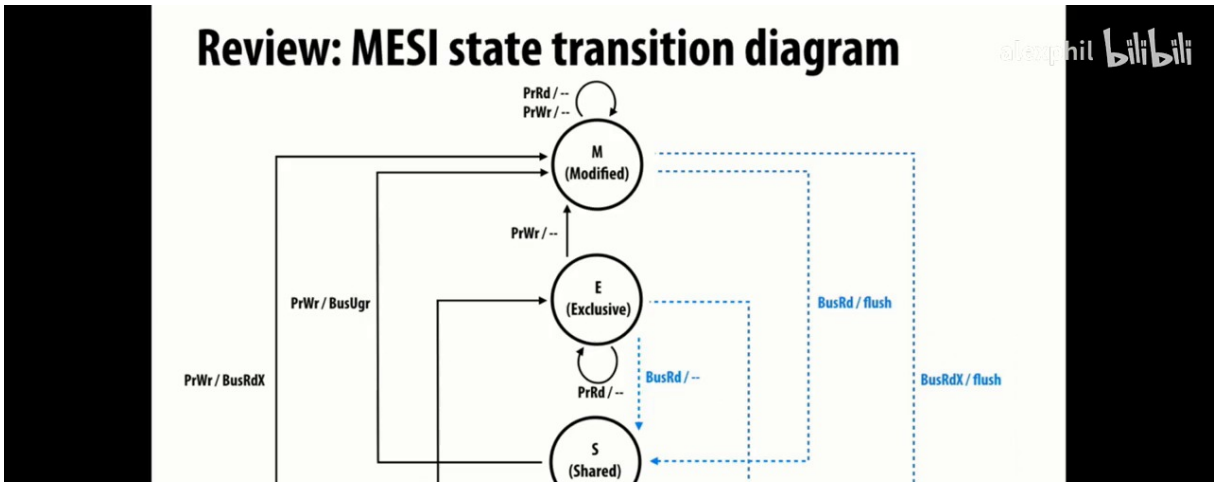


图 3: 回顾：MESI 状态转移图[3]

讲者回顾各状态的含义。独占态表示持有副本且是干净的，同时知道自己是这份数据的唯一持有者。已修改态表示脏副本——被本地缓存写过，与主存中的值不同。

因此若有人需要这份数据，我们可能得做点什么。

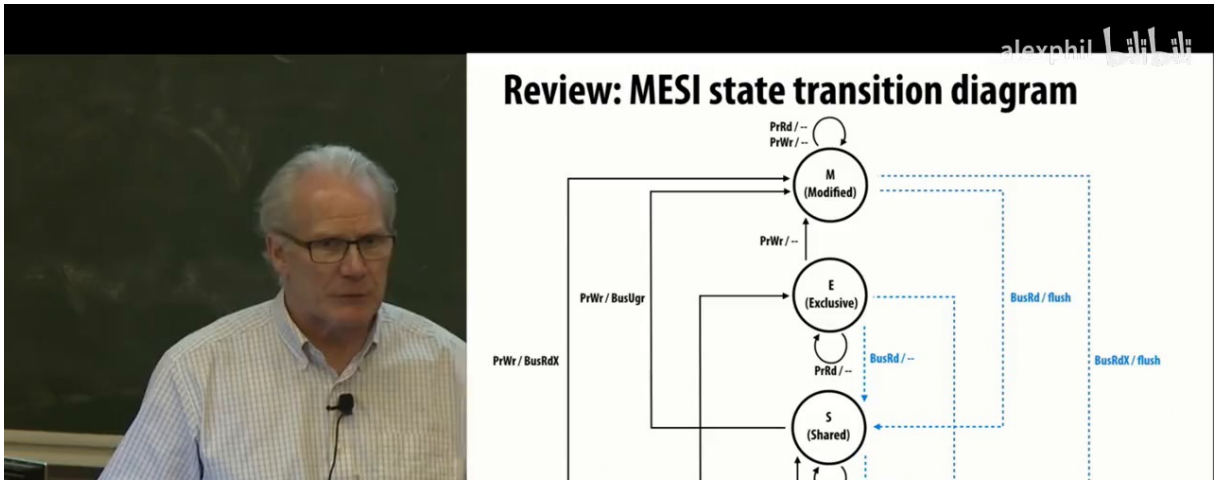


图 4: MESI 状态转移图续：黑线为本地处理器发起，蓝线为系统其余部分发起[4]

讲者解释图中颜色的约定：黑色表示代表本地处理器发生的事务——处理器请求对数据的某种读或写访问；蓝色表示代表系统其余部分发生的事务。

缓存控制器就是那个监听一致性流量、并依据这些缓存命令调整本地状态的部件。

[3] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

[4] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

2.2 多级缓存的现实

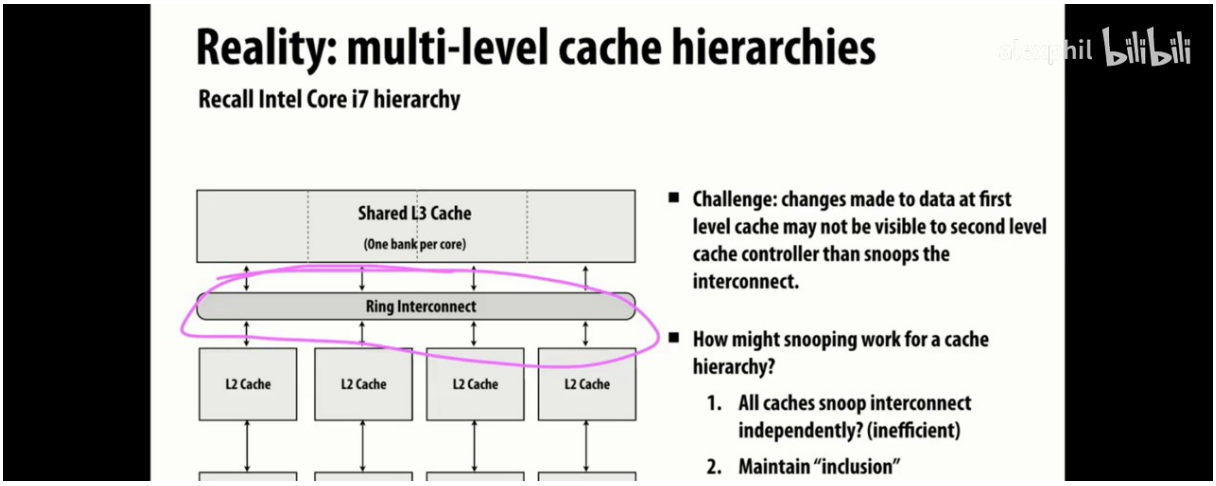


图 5: 现实情形：多级缓存层次，一致性流量发生在互连层[5]

讲者指出真实机器里已经不只是处理器和缓存，而是多级缓存。真正的一致性流量发生在某个互连层次上——它把各个核连到一个共享缓存。

为了讨论方便，他把靠近核的一侧叫缓存，另一侧叫内存。但他提醒：如例子所示，那个”内存”本身可能又是某个更全局状态的缓存。

由此引出一个关键问题：L2 缓存控制器如何能足够了解 L1 里有什么，从而正确地监听并响应总线上的流量?

2.3 本章小结

MESI 的四态语义是：独占是干净且唯一，已修改是脏的。图中黑线为本地处理器发起，蓝线为系统其余部分发起，缓存控制器据此调整本地状态。真实机器是多级缓存，一致性流量发生在互连层，而所谓内存本身可能又是更上层的缓存。由此引出核心问题：L2 如何知道 L1 的内容。

3 包含属性：L2 如何知道 L1 有什么

3.1 包含属性的定义

讲者给出答案：假设存在所谓的包含属性——内层缓存中的任何内容都是外层缓存的子集。

这样 L2 基本上就知道 L1 的全部内容。而且它不仅知道它们在那里，还知道它们的状态：是干净的还是脏的。

[5]视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

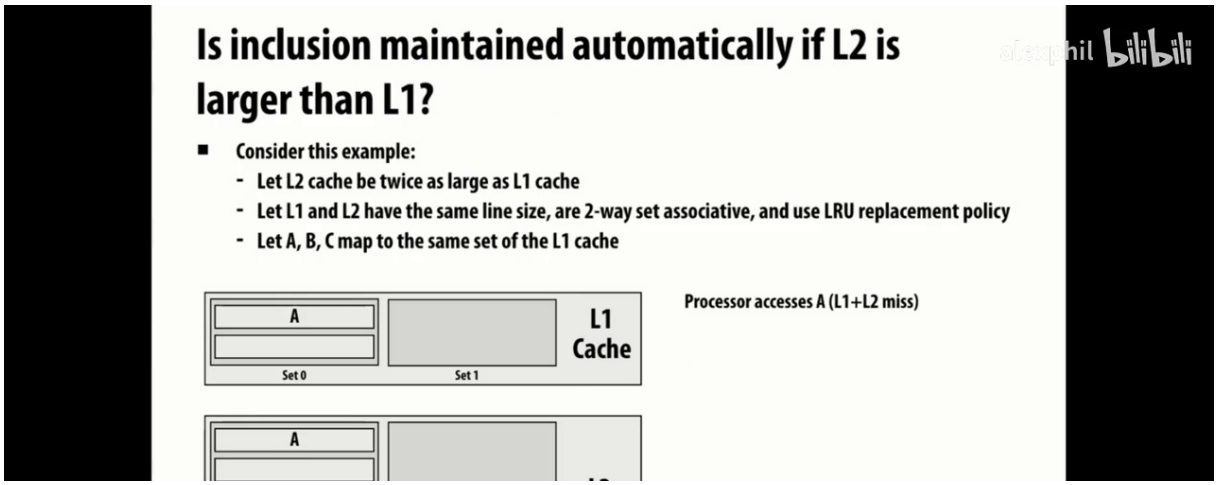


图 6: 若 L2 远大于 L1，包含属性会自动维持吗[6]

$$\mathrm{L1}_{\mathrm{contents}} \subseteq \mathrm{L2}_{\mathrm{contents}}$$

- 内层缓存持有的每一行，外层缓存必须也持有；
- 这样外层才能代替内层响应一致性请求；
- 状态（干净或脏）也必须在外层可见，否则无法判断是否需要取回数据。

3.2 有些实现放弃了包含性

讲者提到有些实现并不满足这个属性：它们允许 L1 缓存与 L2 不同步，保留那些已从 L2 淘汰的行。

他坦承自己不知道它们如何实现这套逻辑，但推测那是协议里又一层复杂度。

3.3 LRU 不保证包含性

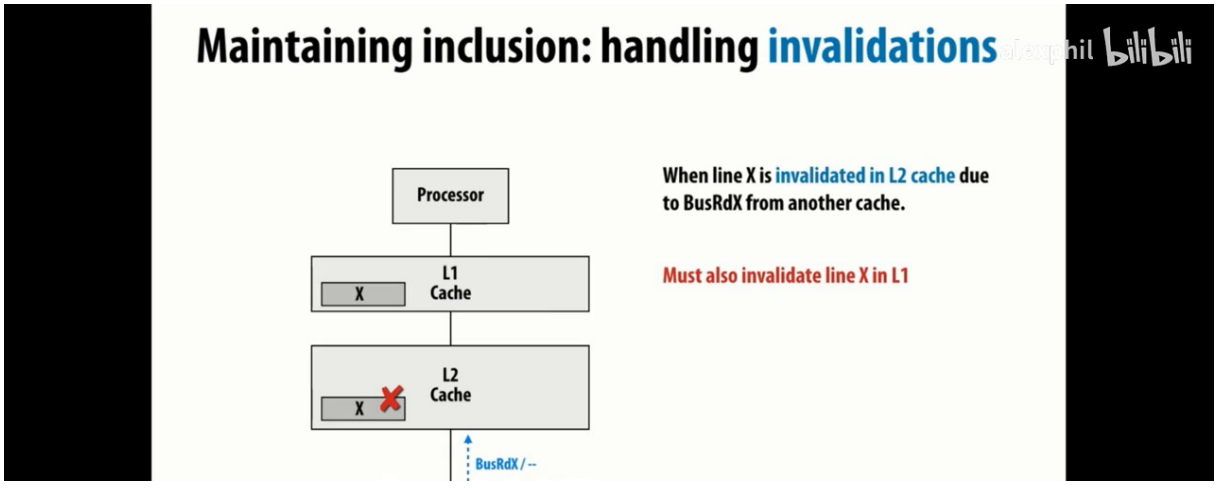


图 7: 维持包含属性：处理失效[7]

[6] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

讲者举了一个具体反例。若两个缓存都用 LRU 策略，可以设想这样的场景：L1 中有大量活动，而这些活动 L2 看不到，因为 L1 自己处理了它们。

于是某个块被从 L2 中淘汰——尽管它在 L1 中非常活跃。

所以标准 LRU 策略并不能保证包含属性。而且这会是相当常见的情形。

因此必须在两个缓存之间人为强制这个策略，主要做法是：若 L2 决定要淘汰某行，就强制 L1 也淘汰它。

包含性是强加的策略而非自然结果

”L2 比 L1 大” 并不自动保证包含性，因为两个缓存各自独立地按 LRU 淘汰。要让包含性成立，必须在硬件里显式耦合两者的淘汰决策：外层淘汰时通知内层一起淘汰。这份耦合本身就是实现复杂度的来源。

3.4 L1 写命中与额外状态位

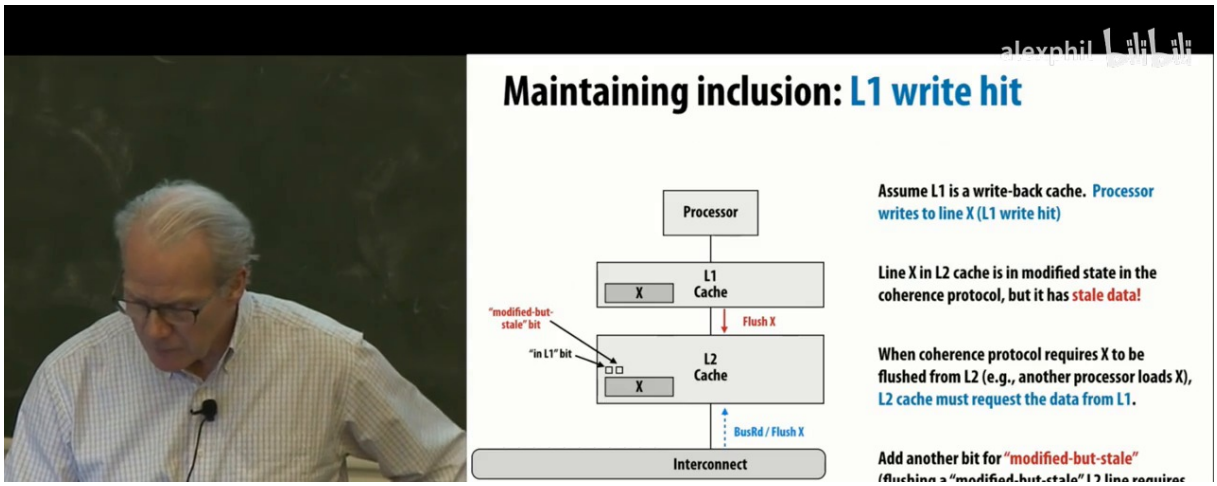


图 8: 维持包含属性：L1 写命中时的处理[8]

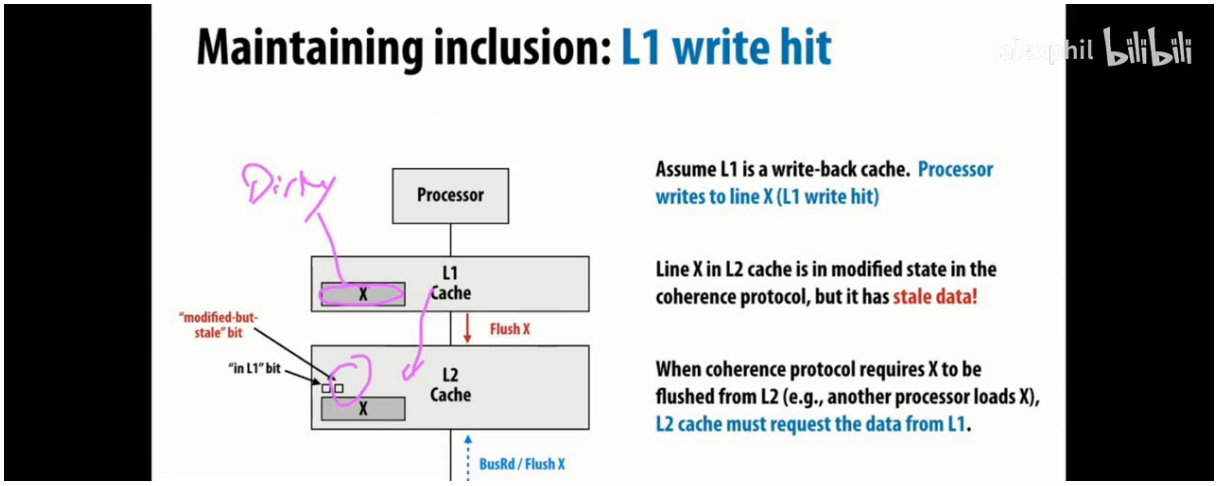


图 9: L1 写命中续：L2 需增加一位表示” 持有但副本不正确”[9]

[7]视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

[8]视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

讲者接着讨论另一种情形：假设 L1 是回写缓存，本地处理器一直在更新这一行，因此它被标记为脏。

他说你不希望每次 L1 写都穿透到 L2，否则就会有写直达缓存的全部问题。所以要让 L1 能持有这份数据并保持脏状态，甚至与 L2 不一致。

于是 L2 缓存里要增加一位额外的状态，表示：这个值确实在缓存中，但我没有它的正确副本。

那么当外部信号说”把你的 x 副本给我”时（比如有人要读），这份信息必须先由 L1 推到 L2，再由 L2 把数据放到总线上。

脏位归属决定了谁负责提供数据

若 L1 持有脏副本而 L2 只有过时值，那么任何外部请求都必须先由 L1 把数据推到 L2 才能应答。这意味着 L1 与 L2 之间会因一致性流量而产生额外通信。协议设计的目标是让这种通信只在必要时发生，否则流量会失控。

3.5 本章小结

包含属性要求 L1 内容是 L2 的子集，这样 L2 才知道 L1 有什么及其状态。有些实现放弃它，代价是额外复杂度。标准 LRU 不保证包含性，必须人为耦合淘汰决策：L2 淘汰时强制 L1 一起淘汰。当 L1 持有脏数据时，L2 需要一位表示”持有但副本不正确”，外部请求需由 L1 推到 L2 再上总线。

4 三个目标与三种失败

4.1 目标

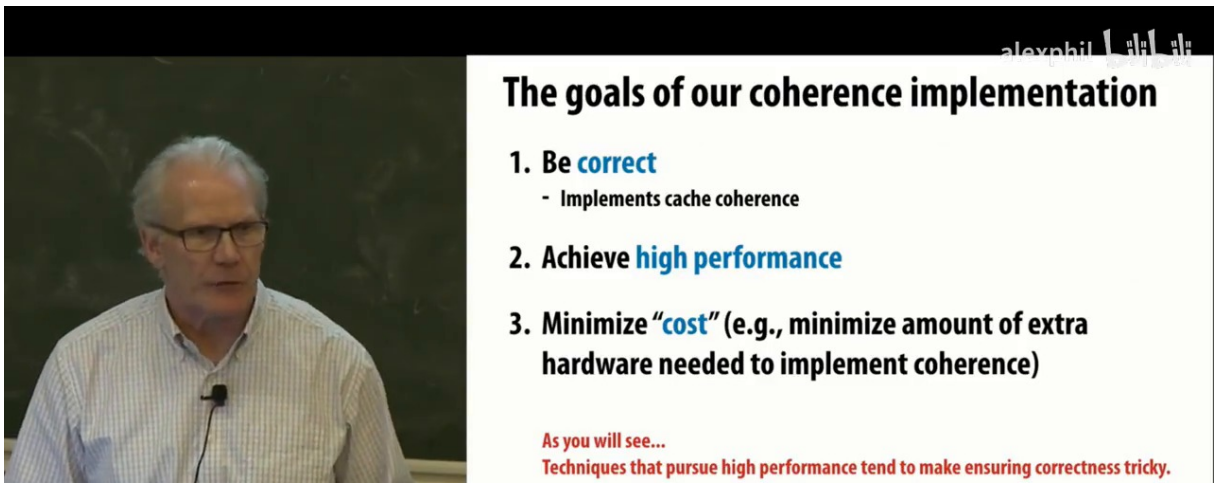


图 10: 一致性实现的目标：正确性与高性能[10]

讲者把目标列出来：第一是实现缓存一致性，也就是正确；第二是获得高性能。

他说这就是全局目标：我们必须正确，这是压倒性的要求，但同时也需要性能。而性能取决于流量有多少、以及不必要的 L1 与 L2 通信有多少。

因此原则是：每个缓存应该只在真正必须时才与其他缓存通信，否则就会增加过多流量。

[9] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

[10] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

正确性与流量是一对必须同时满足的约束

只追求正确可以退化为全部串行；只追求流量最小可以退化为不保证一致。因此协议设计的实质是在”正确”这个硬约束下最小化流量。任何优化都必须先证明不破坏正确性，这也是为什么形式验证在此领域如此重要。

4.2 死锁

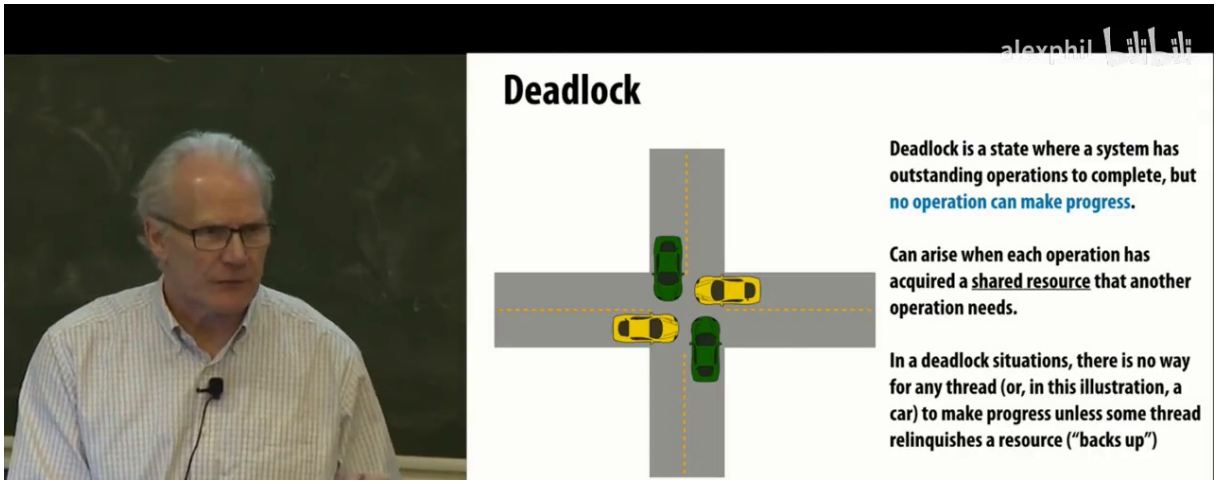


图 11: 死锁：系统有未完成操作，但没有任何操作能推进[11]

讲者给出死锁的定义：系统有未完成的操作，但没有任何操作能向前推进。

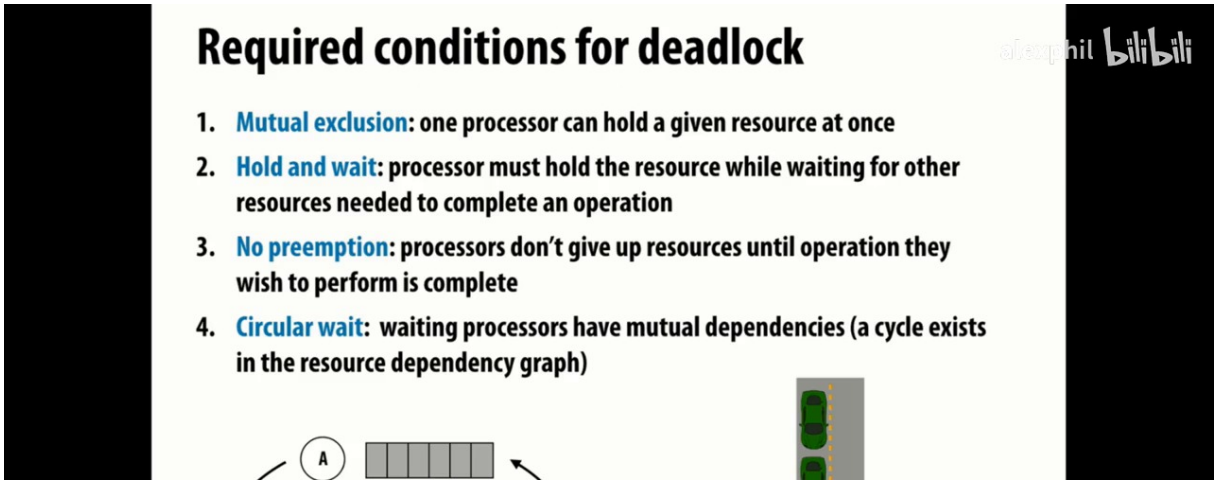


图 12: 死锁的必要条件：互斥、持有并等待、不可剥夺、循环等待[12]

他列出了死锁的必要条件，这是操作系统课上的经典结论：互斥（一个处理器能持有某资源）、持有并等待、不可剥夺、循环等待。四个条件同时成立才会死锁，因此打破任何一个就能避免。

[11] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

[12] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

4.3 活锁

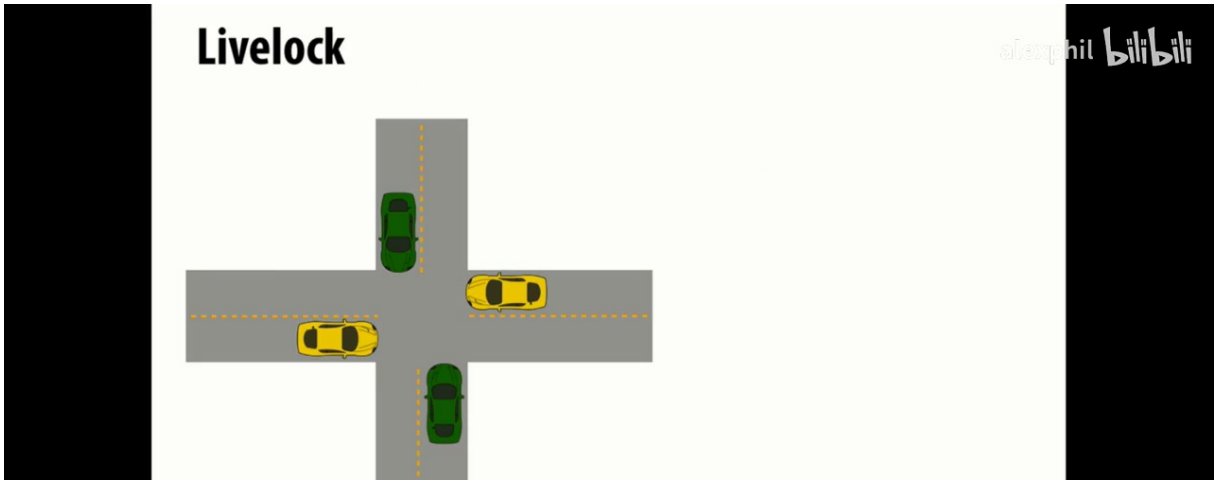


图 13: 活锁：有人在动，但没有任何有效推进[13]

讲者说还有一种较难理解的情况叫活锁。它不是” 什么都没发生”，而是” 发生的事情没有意义”。

他用汽车场景举例：四辆车同时说” 我倒车让你过”，结果没有一辆真正前进。它们同步得太紧，于是来回摆动，谁都无法推进。

死锁与活锁的区别在于” 是否在动”

死锁是所有操作都停住；活锁是操作一直在执行但状态在原地打转。因此监控”CPU 是否繁忙” 无法发现活锁——系统看起来在满负荷工作，实际没有任何进展。诊断活锁需要看” 状态是否有净推进”，而不只是看利用率。

讲者说活锁在现实中较少见，因为事情往往有随机性从而避免它。它可能出现的形式不是永久活锁，而是陷入这种来回摆动、拖慢系统。

4.4 指数退避

讲者提到早期以太网发展出的一个有意思的想法，它出现在很多协议里：指数退避。

用汽车场景想象：不是所有车都同时” 嗖嗖嗖” 地动，而是每辆车退让后等待一段随机时间再尝试前进。这样至少有一辆车能在下一轮通过，而不会所有车同时冲上去。

而” 指数” 的意思是：随机延迟应按指数分布逐渐变长：

$$\text{delay}_k = \text{rand}\left(0,\ 2^k \cdot \tau\right)$$

- k：连续冲突的次数；
- τ：基本时间单位；
- 上限随冲突次数指数增长，因此重试时间被逐渐拉开。

这样多次冲突后，各主体的重试时刻会分散到不同时间窗，避免持续同步碰撞。

[13] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。

4.5 饥饿

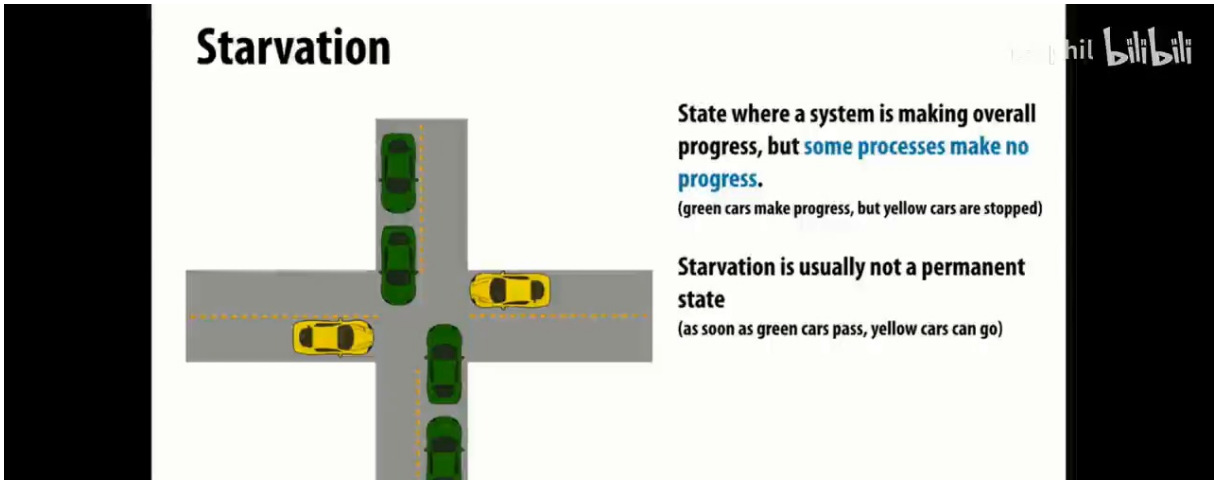


图 14: 饥饿：系统整体在推进，但某些进程毫无进展[14]

讲者区分第三种情况：饥饿。系统整体在取得进展，但某些进程完全没有进展。

他给出具体机制：若有 P 个处理器，而你总是把优先级给编号最小的那个，那么编号大的可能永远轮不到。因此必须建立公平系统——让优先级随机或不断变化，使没有谁会被永远挤出去。

固定优先级是最常见的饥饿来源

按编号、按到达顺序、按固定表项分配优先级的方案，都会在长时间运行后让某些主体长期得不到服务。因此仲裁逻辑必须引入随机化或轮转，代价是平均延迟略高，收益是消除最坏情况的无限等待。

4.6 本章小结

目标是正确且高性能，而性能取决于流量；原则是缓存只在必要时通信。三种失败形态是：死锁（全部停住，需四个必要条件同时成立）、活锁（在动但无有效推进，用指数退避缓解）、饥饿（整体推进但个别永远轮不到，需随机或变化的优先级）。

有了这些概念，就可以讨论具体的设计假设。

[14] 视频画面时间区间：00:20:00–00:20:15。若为裁剪图，时间区间仍对应原视频画面。

5 简化假设与总线事务

5.1 假设

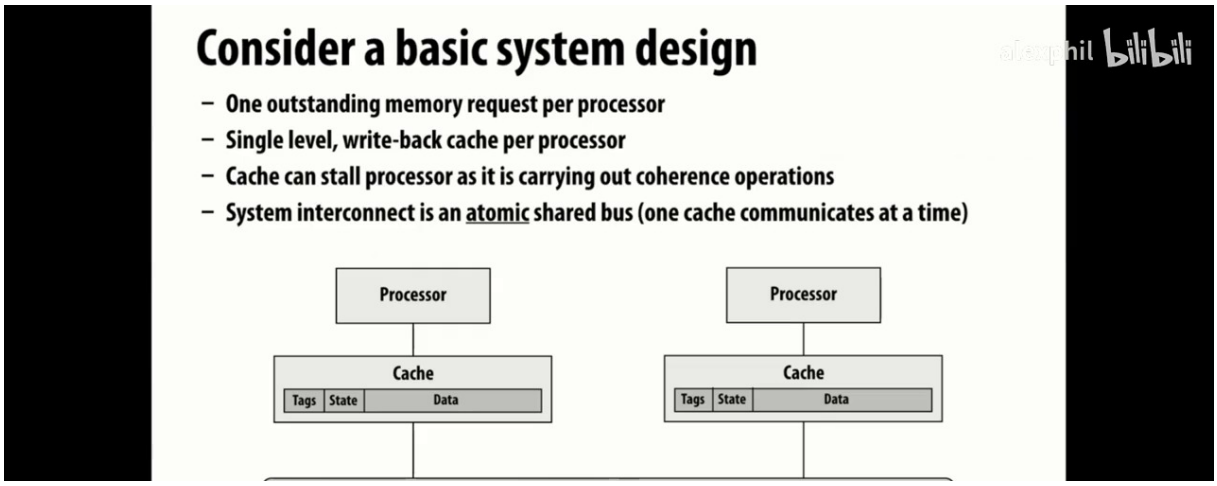


图 15: 基本系统设计：每处理器同时只有一个未完成请求，单级回写缓存[15]

讲者说先把事情简化。假设每个处理器同时只能有一个内存请求，必须完成该事务才允许发出下一个请求。这减少了并发活动的数量与流量规模。

假设只有简单的缓存，而且最初按回写缓存考虑，暂不担心多级缓存。

假设缓存可以回信号给处理器说”等一下，我正在处理一致性流量”。

还假设消息是原子的：总线上一次只有一个消息，它要完成的事务必须原子地完成，中间不插入任何其他活动。

原子性假设是简化，也是限制

原子总线让协议推理只需考虑单事务内部的顺序，不必处理事务交错。但这个假设依赖物理上的共享介质与仲裁机制，一旦改用分组交换互连，原子性就不再成立，协议必须重新设计以处理消息乱序与部分完成。

[15] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

5.2 总线事务的步骤

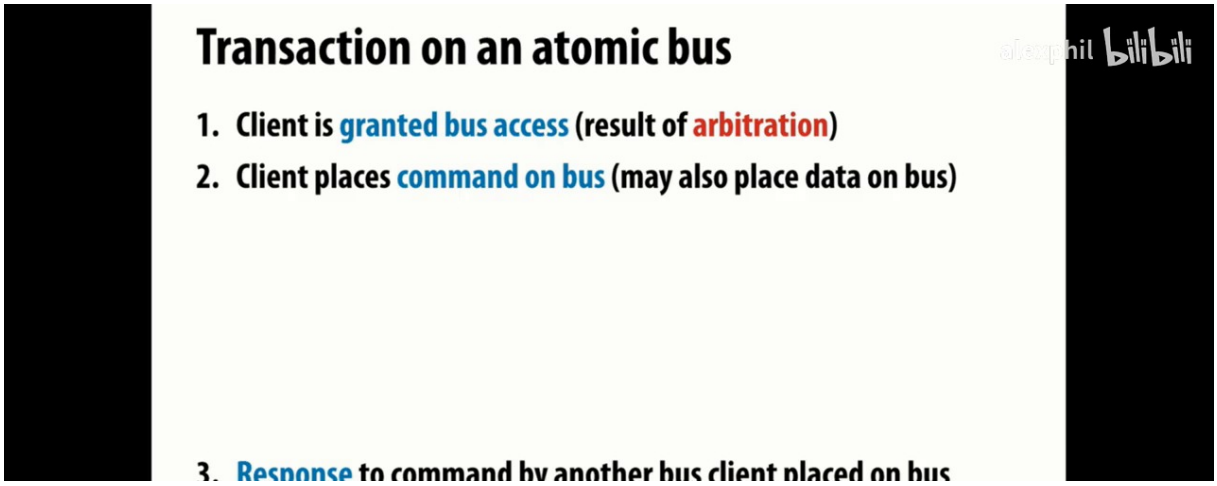


图 16: 原子总线上的事务：获得授权、广播请求、收集回应[16]

讲者描述一次事务：客户端先通过仲裁获得总线访问权，然后把请求广播出去，各缓存回应。

原子总线把并发问题降维成串行问题

若总线保证一次只有一个事务且不被插入，那么协议的正确性论证就只需考虑”单事务内各缓存的回应顺序”，而不必考虑事务之间的交错。这正是早期总线机器的优势，也是可扩展性的代价来源：原子性依赖物理上的共享介质。

5.3 本章小结

简化假设是：每处理器一个未完成请求、单级回写缓存、缓存可让处理器停顿、总线消息原子且不可插入。总线事务的步骤是获得授权、广播、收集回应。原子总线把并发问题降维成串行问题，这也是它难以扩展的原因。

[16] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

6 多处理器控制器与 Snoop Pending

6.1 单处理器逻辑与多处理的差别

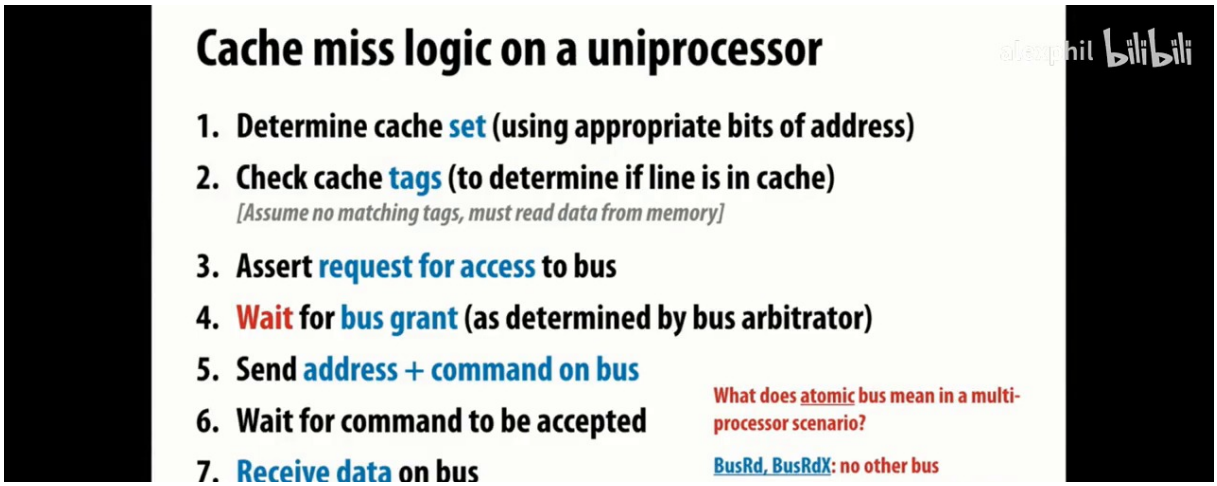


图 17: 单处理器上的缓存缺失逻辑：确定组、检查标签、选择牺牲行[17]

讲者先回顾单处理器上的缺失处理步骤：用地址的相应位确定缓存组，检查标签，若缺失则选择牺牲行。

6.2 核心挑战

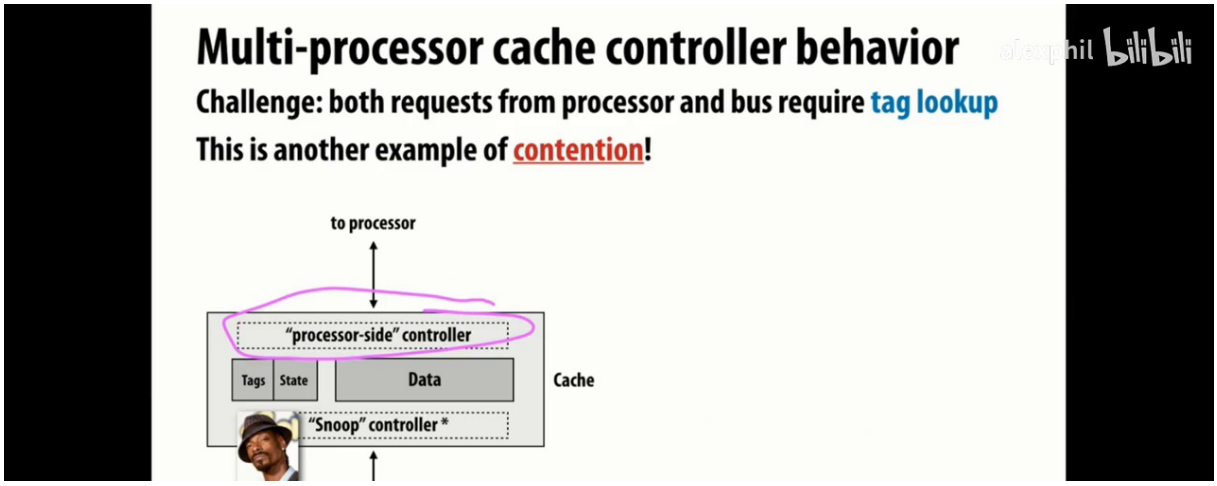


图 18: 多处理器控制器的挑战：处理器请求与总线请求同时需要标签查找[18]

[17] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。
[18] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

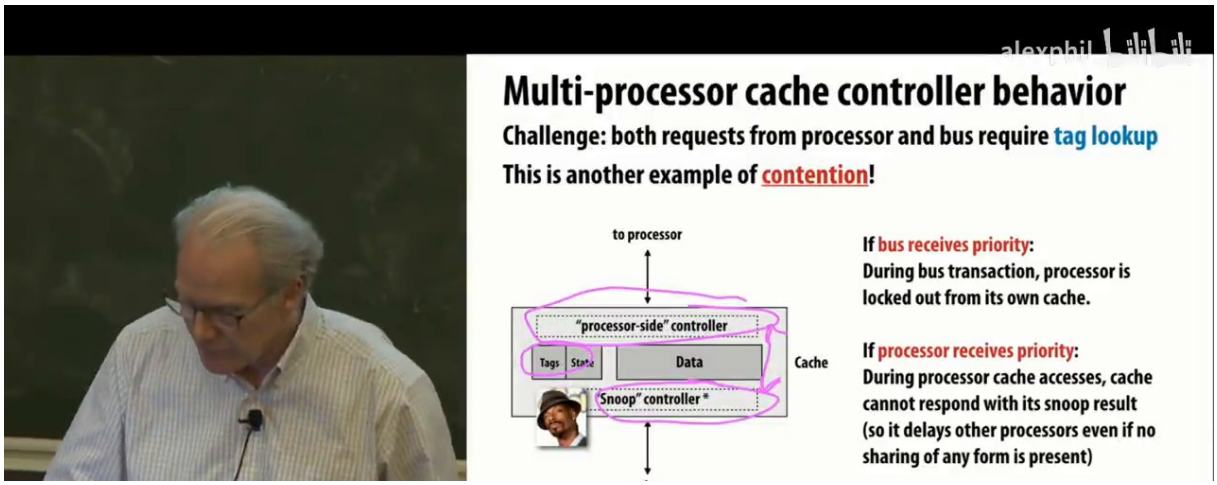


图 19: 挑战续：如何在不冲突的情况下保证事务完整[19]

讲者点出多处理器的核心挑战：来自处理器的请求与来自总线的请求同时需要标签查找。而标签存储器是共享的硬件资源，不能同时服务两者。

一致性的实现代价一半在”资源争用”而非”状态机”

状态机只描述该做什么，而实现必须解决”两件事同时要做时谁先用硬件”。标签查找、缓存读写端口、总线驱动都是单份资源，因此控制器必须排队或复制这些资源。这类冲突在状态图上完全不可见，却是实现复杂度的主要来源。

6.3 Snoop Pending

讲者解释了硬件上的一个具体机制。总线事务一开始，所有缓存会暂时把某条线拉高，或发出信号说”等等，给我机会回应”。

然后当它们各自把消息发出后——无论是共享、脏，还是都不是——就把信号拉低。

一旦这条 pending 线重新变低，下一个动作才能发生。

因此用总线上的线与（OR）逻辑，让所有缓存能集体地表达信息，同时也能跟踪何时可以进入下一步。

Snoop Pending 是硬件级的会合点

它用一根线与线实现了一个分布式屏障：任何一个缓存拉高，全系统就等它。这让”所有监听者都已回应”成为一个可观测的硬件信号，而不需要中央控制器逐个查询。代价是它把最慢者的延迟暴露为全局延迟，与第 9 讲的掉队者问题同构。

6.4 本章小结

单处理器的缺失处理是确定组、查标签、选牺牲行。多处理器的挑战在于处理器请求与总线请求同时需要标签查找，这是状态图上不可见的资源争用。Snoop Pending 用线与逻辑实现分布式会合：任何缓存拉高则全系统等待，全部拉低才进入下一步。

[19] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

7 数据从哪来：事务分裂

7.1 两种数据来源

讲者指出一个有趣的问题：数据可能来自内存，也可能来自另一个缓存。而内存的响应比缓存慢。

于是问题变成：是否应该在还不确定内存里是否有正确数据时就开始响应？

7.2 这是一个设计决策

讲者说这是个有意思的设计决策，因为它会产生额外的内存访问。

$$T_{\text{split}} = T_{\text{probe}} + T_{\text{data}}$$

- T_{probe}：先探测是否有缓存持有该数据所需的时间；
- T_{data}：真正取回数据的时间；
- 若等待探测结果再取，延迟是两者之和；
- 若并行启动，命中时更快，未命中时浪费一次内存访问。

提前启动响应是用带宽换延迟

在不确定数据来源时就启动内存响应，可以让命中缓存的常见情形更快；但若最终发现数据在缓存里，那次内存访问就浪费了。因此这个决策取决于”缓存命中的比例”与”内存带宽的充裕程度”，而不是单纯的延迟优化。

7.3 本章小结

数据可能来自内存或另一缓存，而内存更慢。是否在不确定来源时就启动响应是个设计决策：并行启动降低常见情形延迟，代价是可能浪费内存访问。这个权衡取决于命中比例与带宽充裕度。

讲者在结尾回到一个观察：这类协议让人更体会私有缓存的价值——它们不必为单次访问付出总线流量。

8 总结与延伸

讲者在结尾说，这一讲让人更加体会到缓存的价值：尤其是那些非共享的私有缓存，它们不必为了取一个位置而做那么多总线流量。

把全讲串起来，这是一条从状态图走向实现的路径。状态图只能给出大致概念，实现层面的困难来自并发交错、多级层次、仲裁与超时。工业界认为这类协议极其复杂、极难做对，形式验证在此领域相当成功。真实机器是多级缓存，一致性流量发生在互连层，而所谓内存本身可能又是上层缓存。包含属性（L1 内容是 L2 的子集）让 L2 能代替 L1 响应请求，但标准 LRU 并不保证它——必须显式耦合两者的淘汰决策；当 L1 持有脏数据时，L2 需要额外一位表示”持有但副本不正确”。目标是正确且高性能，而性能取决于流量，因此原则是只在必要时通信。三种失败形态是死锁、活锁与饥饿，分别用打破必要条件、指数退避、随机化优先级来应对。简化假设包括每处理器一个未完成请求与原子总线事务，后者把并发问题降维成串行问题，也是它难以扩展的原因。多处理器的

核心挑战是处理器请求与总线请求同时需要标签查找——这类资源争用在状态图上不可见。Snoop Pending 用线与逻辑实现分布式会合。最后，数据可能来自内存或缓存，是否提前启动响应是一个用带宽换延迟的决策。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 协议正确性必须针对实现论证，而不是针对状态机。状态机假设消息瞬时且无并发，而真实硬件有仲裁延迟、标签争用与超时。因此” 状态图看起来对” 不构成正确性证据，这正是形式验证在此领域必要的原因。
- 包含性是强加的策略，不是缓存大小的自然结果。两个独立按 LRU 淘汰的缓存不保证包含关系，必须显式耦合。理解这一点，才能理解为什么多级一致性协议比单级复杂得多。
- 死锁、活锁、饥饿是三种不同故障，诊断方法也不同。死锁看” 是否停住”，活锁看” 状态是否有净推进”，饥饿看” 个体是否长期得不到服务”。只看 CPU 利用率会把活锁误判为正常工作。
- 事务分裂的取舍取决于负载特征而非纯延迟。提前启动响应在命中率高时有利，在带宽紧张时有害。因此同一协议在不同工作负载下的最优策略可能不同。

这堂课的边界在于它是实现导论：讲者明确说真实协议极其复杂，本讲只给出几类典型问题的形态与对策，不含完整协议规范。

两个可以直接从本讲数据提出的下一步问题：其一，Snoop Pending 把最慢监听者的延迟暴露为全局延迟，那么在监听者数量增加时，这个会合机制如何成为可扩展性的瓶颈？其二，若放弃包含属性以允许 L1 保留已从 L2 淘汰的行，协议需要增加哪些机制才能正确响应总线请求？